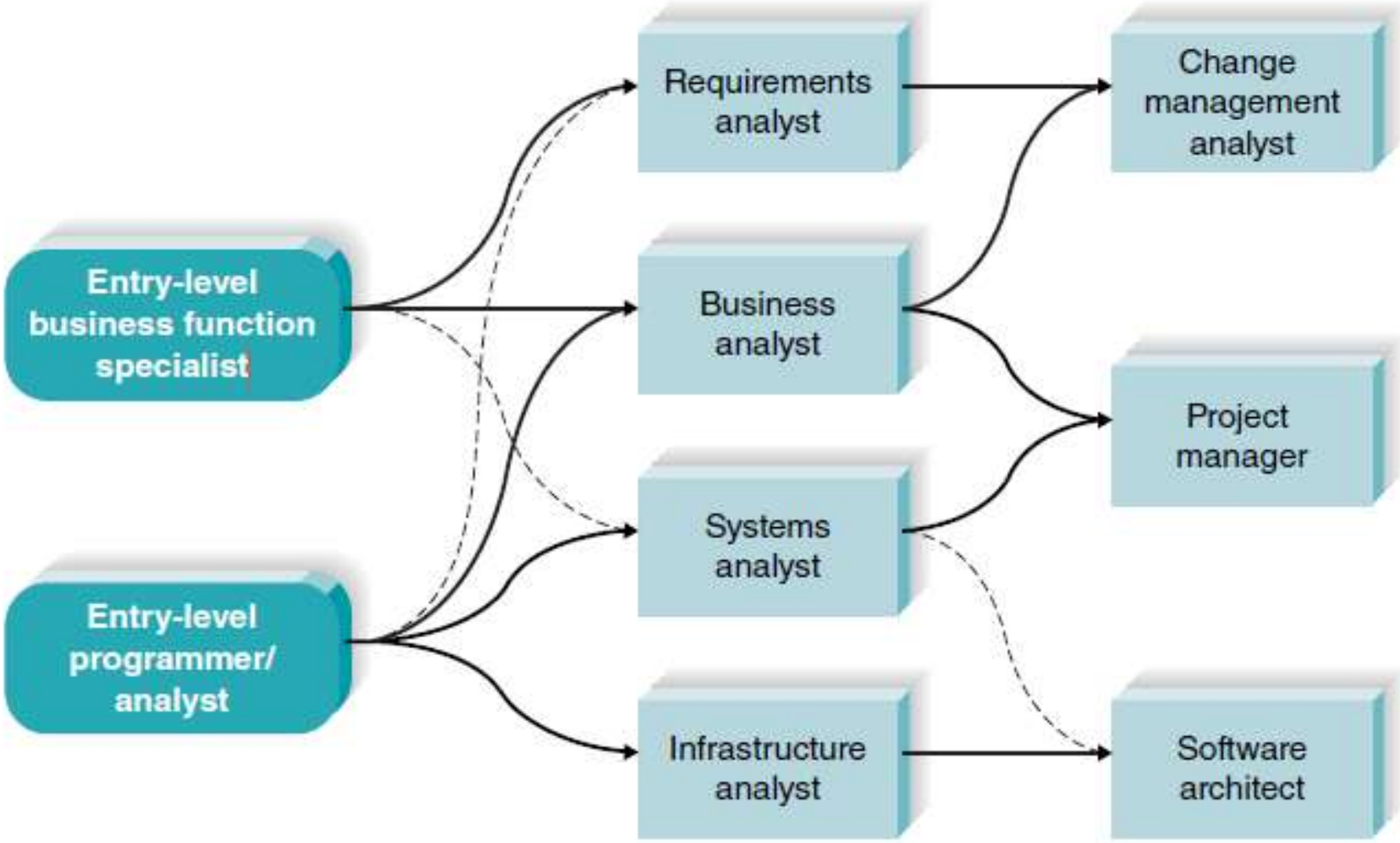


System Analysis

DR. DRAGUNOV

System development methodologies

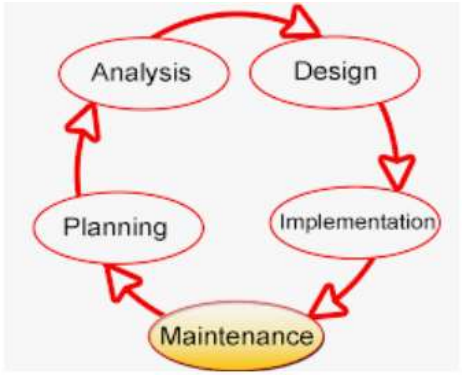
Career Paths for System Developers



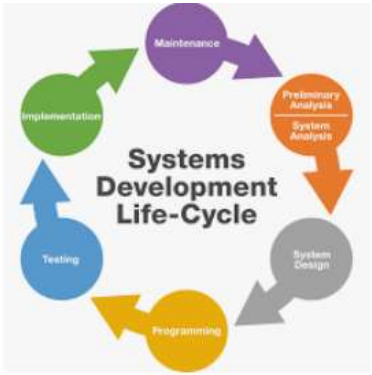
Diversity of SDLC variants



System Development Life Cycle
svitla.com



Systems development life cycle - Wik...
en.wikipedia.org



7.3. Systems Development Li...
ecampusontario.pressbooks.pub



Systems Development Life Cycle. The s...
medium.com



Ultimate Guide to System Develo...
smartsheet.com



System Development Life Cycle (...
researchgate.net



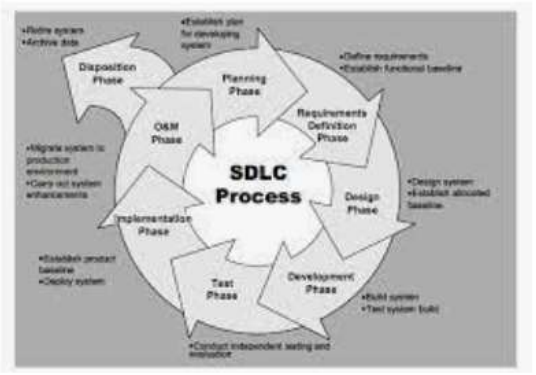
System Development Life Cyc...
bndsbon.wordpress.com



SDLC: Seven Phases of...
innovativearchitects.com



Software Development Life C...
bigwater.consulting



System Development Life Cycle | The IT Law...
itlaw.fandom.com



System Development Life ...
sites.google.com

The solution (at least some help).



Planning – the initial stage in which key business needs supposed to be identified and development methodologies are chosen. Originally, project plan supposed to be developed in planning phase but at this point due to lack of detailed business requirements only high-level plan may be developed. Later with deeper understanding of the systems problem domain and due to scope creep this high-level plan should be refined. Low-level planning should be done regularly depends on chosen development methodology.

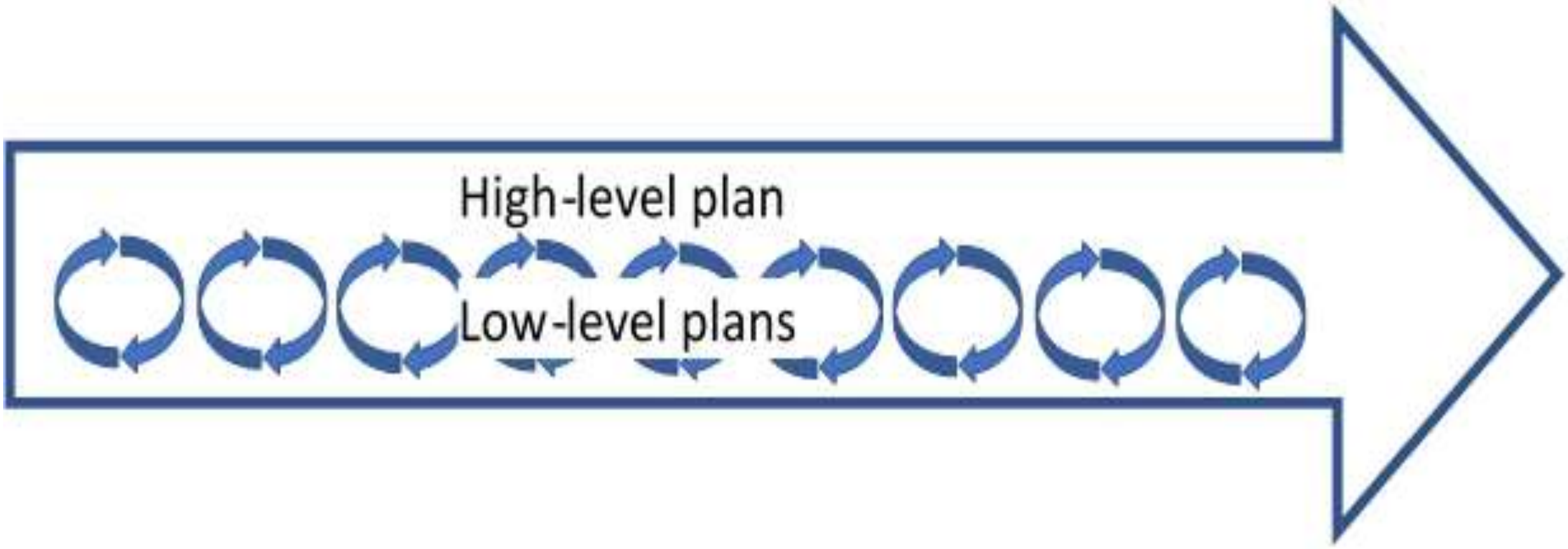
Analysis – is the second, probably, the key phase is aimed on user (or business) requirements gathering and requirements elicitation. Both functional and nonfunctional requirements should be clearly understood at this moment. Also, functional, Structural and Behavioral models may be crated at this stage. At the end of Analysis phase System proposal must be prepared and approved by steering committee of organization.

Design – the third phase when business requirements supposed to be transformed in to form of system requirements. And architecture of the system including subsystems should be prepared.

The fourth stage – Implementation (or Development) phase should include writing the source code, testing and deployment or delivering to end-users.

Last fifth stage is Post Production phase includes maintenance. This is probably the longest phase in SDLC includes continues user training and improvements of the system.

About plans.



Plan-Driven methodology used for high-level planning and Agile methodologies for low-level planning.

SYSTEM ANALYST SKILLS

Skill - the ability to do something well.

Hard skills are professional skills that can be measured, assessed and verified. Hard skills can be confirmed by a specific action, certificate, portfolio.

Examples of hard skills are the ability to drive a car, touch-type, program in a certain programming language.

Soft skills are a person's socio-psychological or personal skills that do not depend on the profession.

It is difficult to measure or confirm them at the moment (at an interview). Soft skills are most often aimed at oneself or at other people. Such skills include the ability to plan your time and concentrate on what is important, analytical thinking, the ability to justify your decisions, the ability to work in a team and teach others, and everything that we often call personal qualities. They can be either innate or acquired through experience.

Determine which statements are true for describing soft skills, and which are true for hard skills.

1. Not related to a specific profession. **Soft skills**

2. Take a long time and are difficult to develop. Mastering them requires a lot of analysis of your own and other people's behavior. **Soft skills**

3. Relatively easy to obtain through training. **Hard skills**

4. Mastery is revealed over time. **Soft skills**

5. Are tied to a specific profession. **Hard skills**

6. Can be confirmed here and now. You can get a certificate or diploma. **Hard skills**

AI-Augmented Analyst – Additional Skills

New Skill	Why It Matters	How AI Helps Develop It
Prompt engineering	Communicate effectively with AI tools	Practice writing structured prompts for requirement extraction
AI output validation	Detect hallucinations, bias, and errors	Compare AI output to manual analysis; identify discrepancies
Data literacy	Understand data requirements for AI components	Ask AI: "What data would I need to build a predictive maintenance system?"
Ethical AI awareness	Identify bias, privacy, fairness issues	Use AI bias detection tools; prompt: "What biases could emerge in a hiring recommendation system?"
AI-tool orchestration	Chain multiple AI tools together	Use ChatGPT to generate requirements → Copilot to generate code → AI tester to generate tests
Model evaluation	Assess if an AI component is "good enough"	Ask AI: "What metrics would you use to evaluate a fraud detection model? Explain precision vs. recall."

What Is an AI-Augmented Analyst?

An AI-augmented analyst is a human analyst who **uses AI tools as cognitive partners** – not replaced by AI, but enhanced by it. The AI handles routine, pattern-based, or data-intensive tasks while the human focuses on judgment, creativity, ethics, and stakeholder interaction.

Division of Labor: Human vs. AI

Task	Human Analyst	AI-Augmented Analyst (with AI)
Interview stakeholders	Conducts interview, reads body language	AI transcribes, summarizes, extracts requirements
Requirements prioritization	Makes final judgment	AI suggests priorities based on historical data
Feasibility analysis	Interprets business constraints	AI generates cost/time estimates from historical projects
Modeling (UML diagrams)	Reviews and corrects	AI generates draft diagrams from text
Documentation	Writes executive summaries	AI generates first draft, human polishes
Risk identification	Evaluates business/political risks	AI identifies technical risks from patterns
Ethical review	Makes ethical judgments	AI flags potential bias or privacy violations

What Makes Design "AI-Aware"?

AI-aware design means designing a system knowing that **AI components have unique characteristics** that affect object-oriented modeling.

Traditional OO Assumption	AI Reality	Design Implication
Methods always produce same output for same input	AI models are probabilistic (different outputs possible)	Add confidence scores, allow retries
State is explicitly managed	AI models have internal state (weights, biases) that are not easily inspectable	Need model versioning, snapshotting
Errors are deterministic bugs	AI errors are statistical (false positives/negatives)	Add human-in-the-loop fallbacks
Performance is predictable	AI inference time varies (batch vs. real-time)	Design for async, caching, batching
Testing is exhaustive	Testing AI requires statistical validation (precision, recall, F1)	Add model evaluation pipelines

AI-Aware OO Design Patterns

Pattern	Description	UML Representation
Model Wrapper	Class that encapsulates an AI model, hiding complexity	Class ChatbotModel with methods predict() and get_confidence()
Human-in-the-Loop	Class that routes low-confidence predictions to humans	Class ReviewQueue with escalate_to_human()
Model Version Manager	Class that handles switching between model versions	Class ModelRegistry with get_model(version)
Feature Store	Class that manages input features for AI models	Class FeatureStore with get_features(entity_id)
Drift Detector	Class that monitors model performance over time	Class DriftMonitor with check_drift()

SYSTEMS DEVELOPMENT METHODOLOGIES

A ***methodology*** is a formalized approach to implementing the SDLC (i.e., it is a list of steps and deliverables).

There are many different systems development methodologies, and each one is unique, based on the order and focus it places on each SDLC phase. Some methodologies are formal standards used by government agencies, whereas others have been developed by consulting firms to sell to clients.

Many organizations have their own in-house developed methodologies that explain exactly how each phase of the SDLC is to be performed in that company. In other cases, methodologies are obtained from consulting fi rms for their clients to follow; provided by the vendor of the soft ware to be installed; or mandated as a part of projects involving government agencies.

Important factor in categorizing methodologies is the sequencing of the SDLC phases and the amount of time and effort devoted to each.

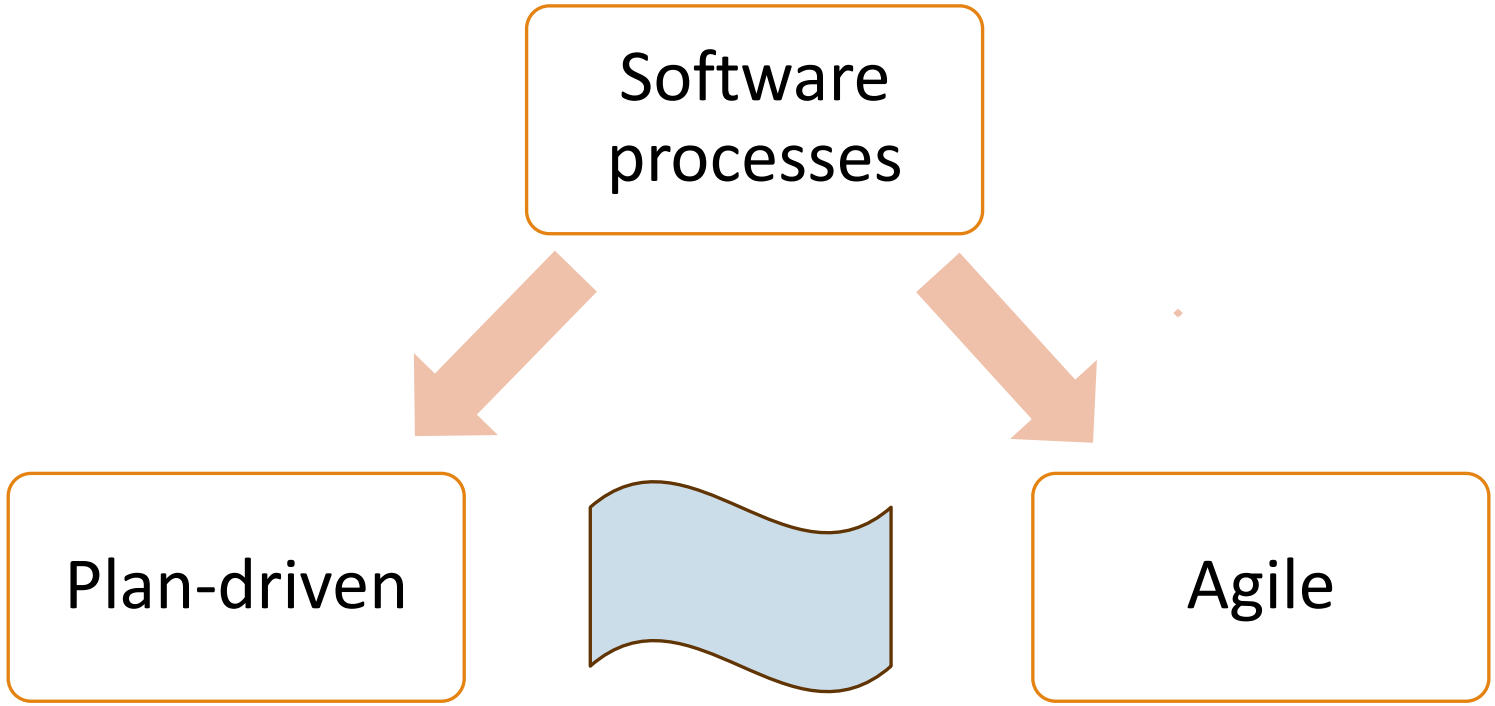
In the early days of computing, programmers did not understand the need for formal and well-planned life-cycle methodologies.

They tended to move directly from a very simple planning phase right into the construction step of the implementation phase—in other words, from a very fuzzy, not-well-thought-out system request into writing code. This is the same approach that you sometimes use when writing programs for a programming class.

list of project characteristics that will affect the methodology selection decision.

- **Clarity of User Requirements** How well do the users and analysts understand the functions and capabilities needed from the new system?
- **Familiarity with Technology** How much experience does the project team have with the technology that will be used?
- **System Complexity** How much complexity is anticipated in the new system? Does the new system include a wide array of features? Will the system have to integrate with many existing systems? Does it span multiple organizational units, or even multiple organizations?
- **System Reliability** Will this system need to be highly reliable or is some downtime tolerable?
- **Short Time Schedules** Is the project time frame tight?
- **Schedule Visibility** Are the project sponsors, users, or organizational managers anxious [ˈaNG(k)SHəs] to see progress?

Software processes can be improved by process standardization where the diversity in software processes across an organization is reduced. This leads to improved communication and a reduction in training time, and makes automated process support more economical.



Plan-driven and agile processes

Plan-driven processes are processes where all of the process activities are planned in advance and progress is measured against this plan.

In **agile processes**, planning is incremental and it is easier to change the process to reflect changing customer requirements.

In practice, most practical processes include elements of both plan-driven and agile approaches.

There are no right or wrong software processes.

Classes of Methodologies

Structured Development

- Waterfall Development
- Parallel Development

Rapid Application Development

- Phased
- Prototyping

Agile Development

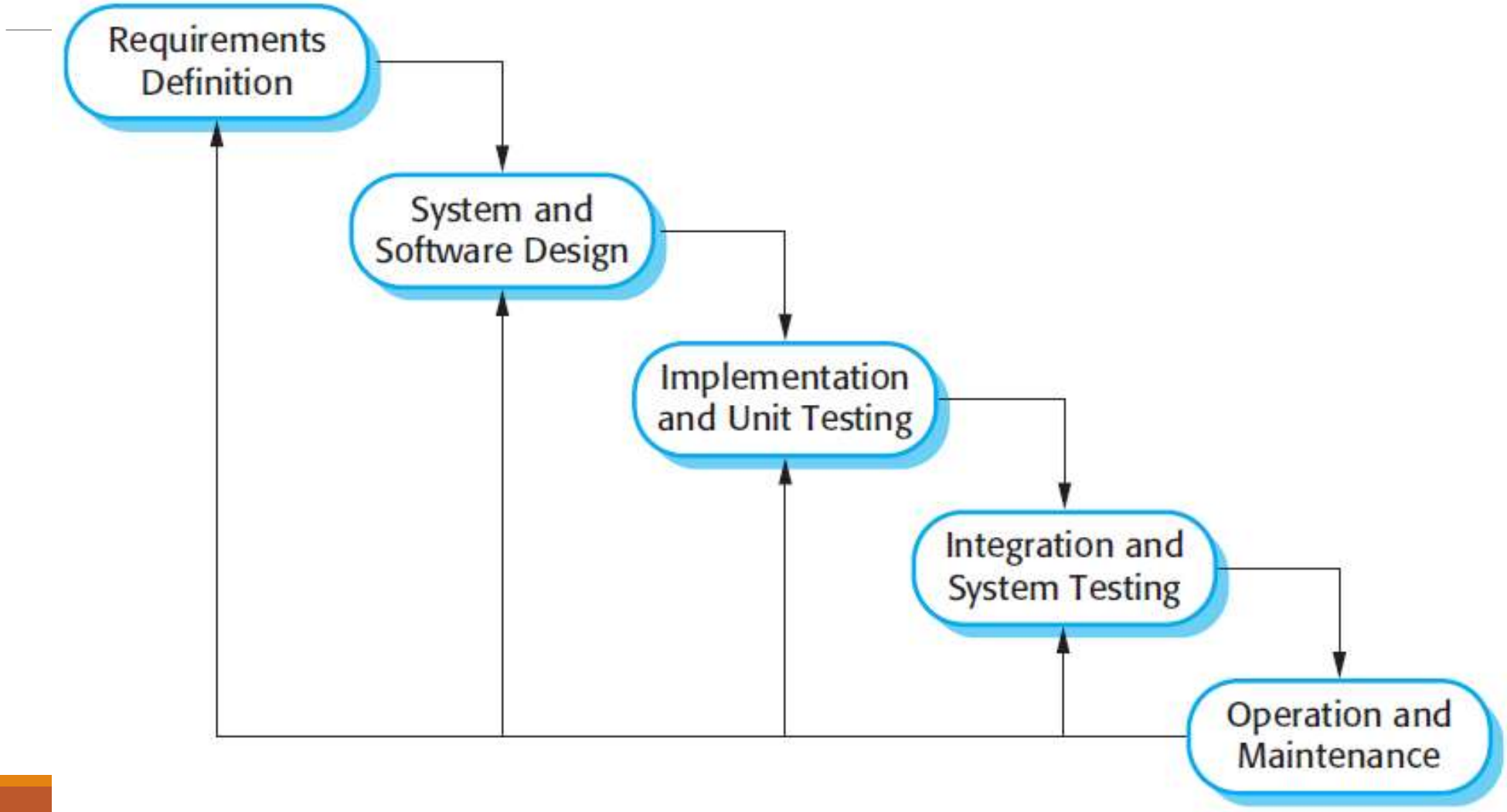
- eXtreme Programming
- SCRUM

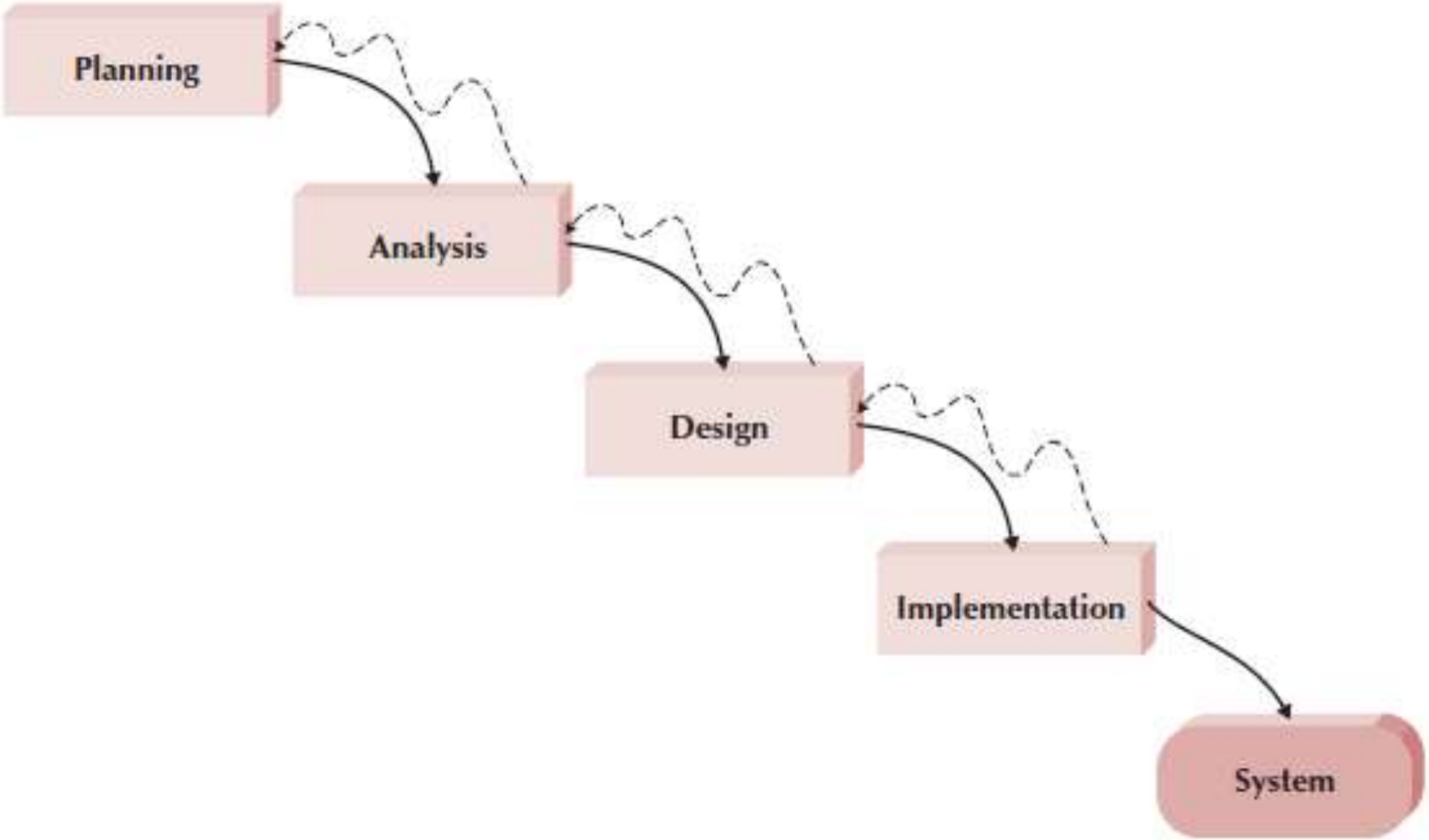
Structured Design

The first category of systems development methodologies is called *structured design.* These methodologies became dominant in the 1980s, replacing the previous ad hoc and undisciplined approach. Structured design methodologies adopt a formal step-by-step approach to the SDLC that moves logically from one phase to the next.

The waterfall model

The first published model of the software development process was derived from more general system engineering processes (Royce, 1970).

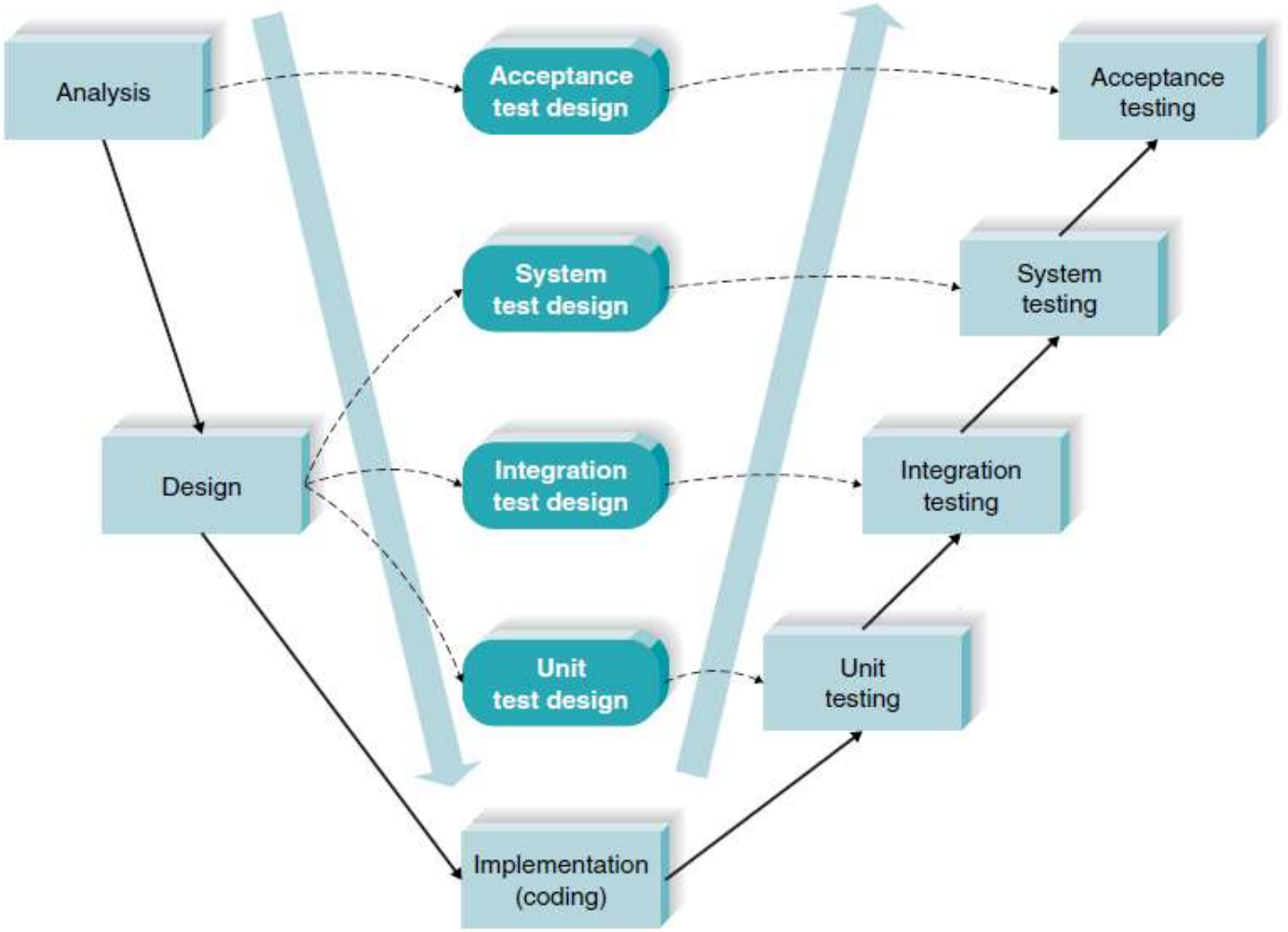



Planning
Analysis
Design
Implementation
System

Because of the cascade from one phase to another, this model is known as the '**waterfall model**' or software life cycle. The waterfall model is an example of a plan-driven process—in principle, you must plan and schedule all of the process activities before starting work on them.



V-model



The development process proceeds down the left –hand slope of the V, defi ning requirements and designing system components. At the base of the V, the code is written. On the upward-sloping right side of the model, testing of components, integration testing, and, fi nally, acceptance testing are performed.

Waterfall model problems

Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements.

- Therefore, this model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process.
- Few business systems have stable requirements.

The waterfall model is **mostly used for large systems engineering projects** where a system is developed at several sites.

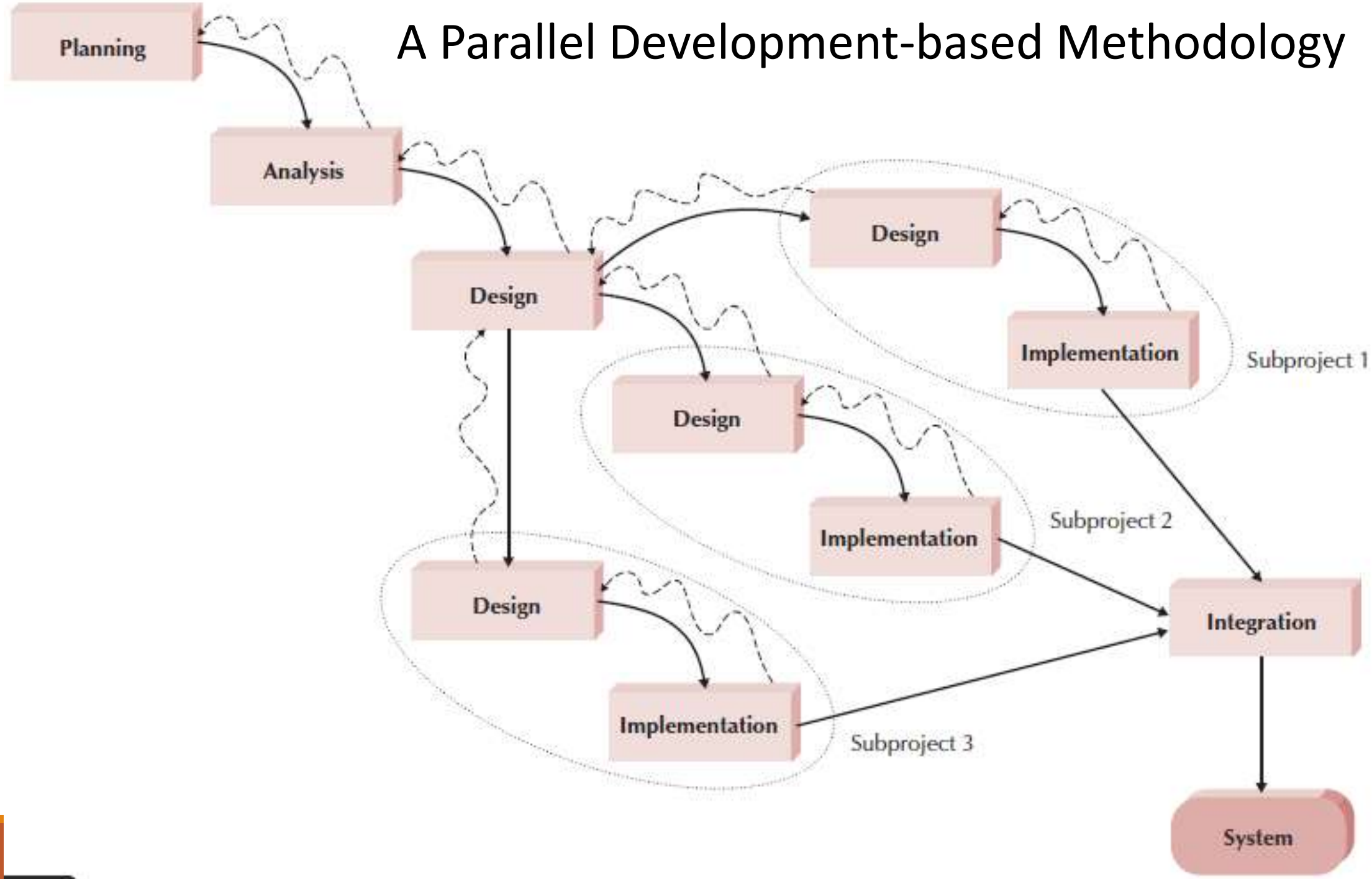
- In those circumstances, the plan-driven nature of the waterfall model helps coordinate the work.

Parallel Development

Parallel development methodology attempts to address the problem of long delays between the analysis phase and the delivery of the system. Instead of doing design and implementation in sequence, it performs a general design for the whole system and then divides the project into a series of distinct subprojects that can be designed and implemented in parallel. Once all subprojects are complete, the separate pieces are integrated and the system is delivered

The primary advantage of this methodology is that it can reduce the time to deliver a system; thus, there is less chance of changes in the business environment causing rework.

A Parallel Development-based Methodology



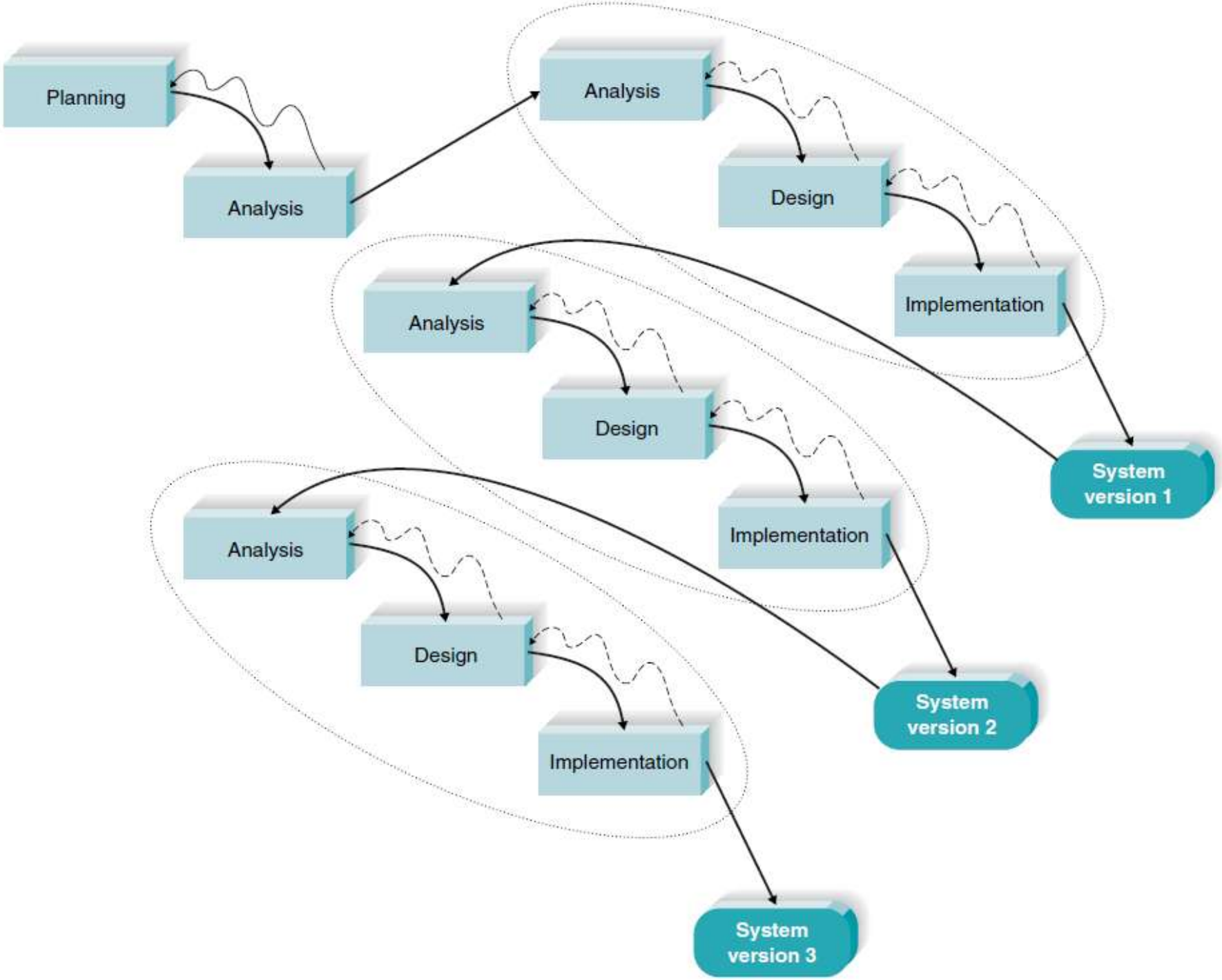
Rapid Application Development (RAD)

A second category of methodologies includes rapid application development (RAD)-based methodologies. These are a newer class of systems development methodologies that emerged in the 1990s. RAD-based methodologies attempt to address both weaknesses of structured design methodologies by adjusting the SDLC phases to get some part of the system developed quickly and into the hands of the users. In this way, the users can better understand the system and suggest revisions that bring the system closer to what is needed.

Most RAD-based methodologies recommend that analysts use special techniques and computer tools to speed up the analysis, design, and implementation phases, such as computer- aided software engineering (CASE) tools, joint application design (JAD) sessions, fourth-generation or visual programming languages that simplify and speed up programming (e.g., Visual Basic), and code generators that automatically produce programs from design specifications.

A Phased (iterative) Development-based Methodology

A phased development-based methodology breaks an overall system into a series of versions that are developed sequentially. The analysis phase identifies the overall system concept, and the project team, users, and system sponsor then categorize the requirements into a series of versions. The most important and fundamental requirements are bundled into the first version of the system. The analysis phase then leads into design and implementation—but only with the set of requirements identified for version 1 Once version 1 is implemented, work begins on version 2. Additional analysis is performed based on the previously identified requirements and combined with new ideas and issues that arose from the users' experience with version 1. Version 2 then is designed and implemented, and work immediately begins on the next version. This process continues until the system is complete or is no longer in use.


Planning
Analysis
Analysis
Design
Implementation
Analysis
Design
Implementation
Analysis
Design
Implementation
System version 1
System version 2
System version 3

There are several approaches to the iterative model, the most popular of which are:

- Incremental
- Iterative
- iterative-incremental

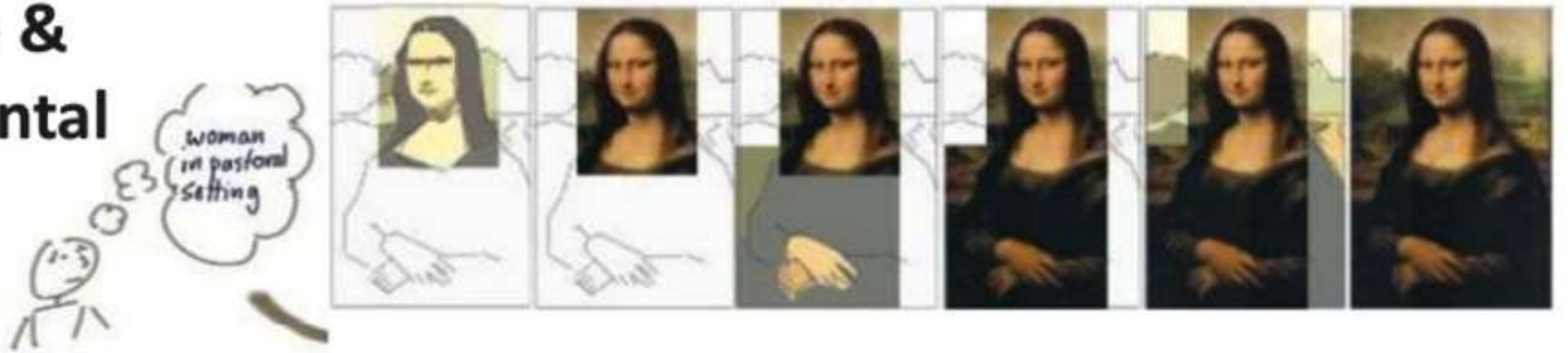
Iterative



Incremental



Iterative & Incremental



Advantages

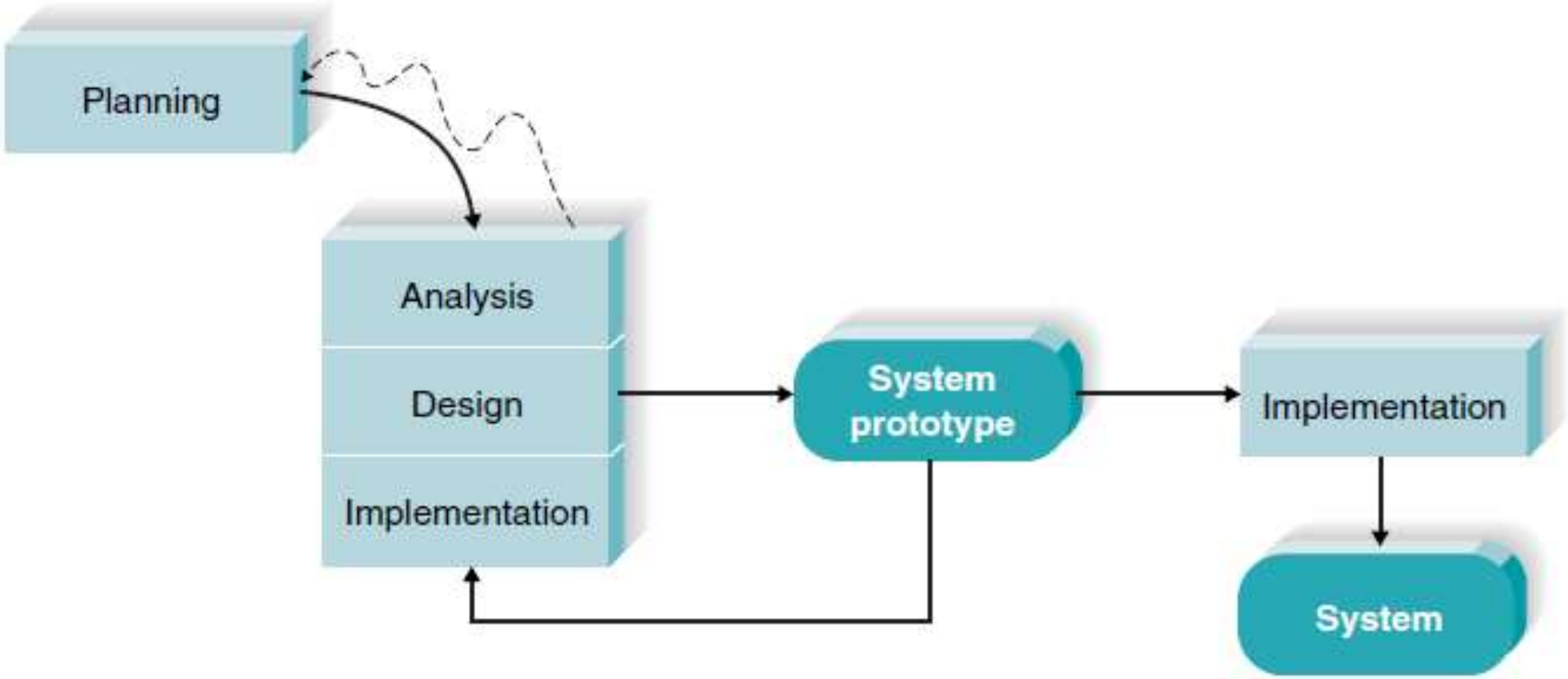
Phased development–based methodologies have the advantage of **quickly getting a useful system into the hands of the users**. Although the system does not perform all the functions the users need at first, it does begin to provide business value sooner than if the system were delivered after completion, as is the case with the waterfall and parallel methodologies. Likewise, because users begin to work with the system sooner, they are more likely to identify important additional requirements sooner than with structured design situations.

Drawback

The major drawback to phased development is that users begin to work with systems that are intentionally incomplete. It is critical to identify the most important and useful features and include them in the first version and to manage users' expectations along the way.

Prototyping

A prototyping-based methodology performs the analysis, design, and implementation phases concurrently, and all three phases are performed repeatedly in a cycle until the system is completed. With these methodologies, the basics of analysis and design are performed, and work immediately begins on a system prototype, a quick-and-dirty program that provides a minimal amount of features.


Planning
Analysis
Design
Implementation
System prototype
Implementation
System

Prototyping

A **prototype** is an initial version of a system used to demonstrate concepts and try out design options.

A prototype can be used in:

- The **requirements engineering** process to help with requirements elicitation and validation;
- In **design** processes to explore options and develop a UI design;
- In the **testing** process to run back-to-back tests.

Benefits of prototyping

- **Improved** system usability.
- A closer match to users' real needs.
- Improved design quality.
- Improved maintainability.
- Reduced development effort.

Prototype development

May be based on rapid prototyping languages or tools

May involve leaving out functionality

- Prototype should focus on areas of the product that are not well-understood;
- Error checking and recovery may not be included in the prototype;
- Focus on functional rather than non-functional requirements such as reliability and security

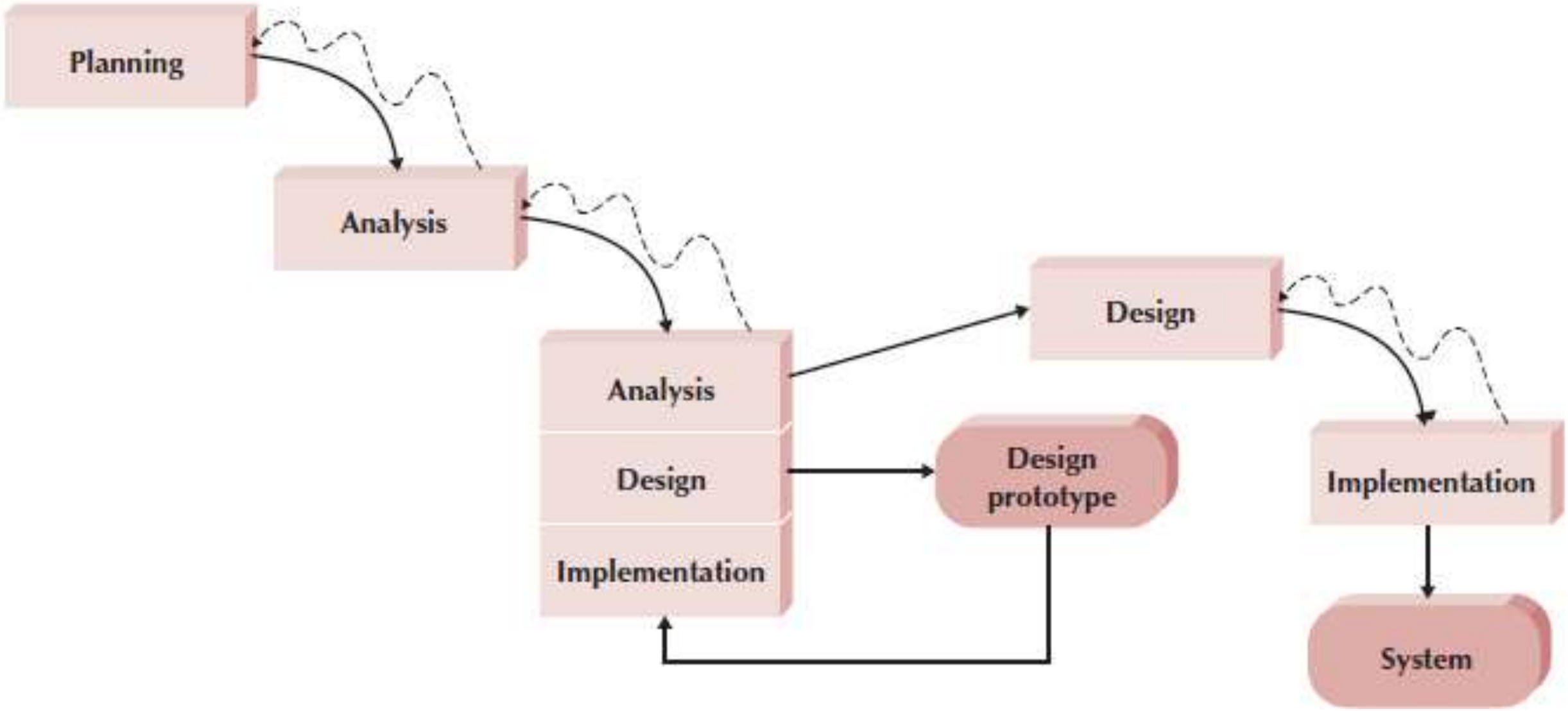
Throwaway prototyping

Throwaway prototyping-based methodologies are similar to prototyping-based methodologies in that they include the development of prototypes; however, throwaway prototypes are done at a different point in the SDLC. These prototypes are used for a very different purpose than those previously discussed, and they have a very different appearance

Prototypes should be discarded after development as they are not a good basis for a production system:

- It may be impossible to tune the system to meet non-functional requirements;
- Prototypes are normally undocumented;
- The prototype structure is usually degraded through rapid change;
- The prototype probably will not meet normal organizational quality standards.

A Throwaway Prototyping–based Methodology



Agile Development

Agile development is a group of programming-centric methodologies that focus on streamlining the SDLC. Much of the modeling and documentation overhead is eliminated; instead, face-to-face communication is preferred. A project emphasizes simple, iterative application development in which every iteration is a complete soft ware project, including planning, requirements analysis, design, coding, testing, and documentation. Cycles are kept short (1–4 weeks), and the development team focuses on adapting to the current business environment.

Agile Development

Agile manifesto

We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:

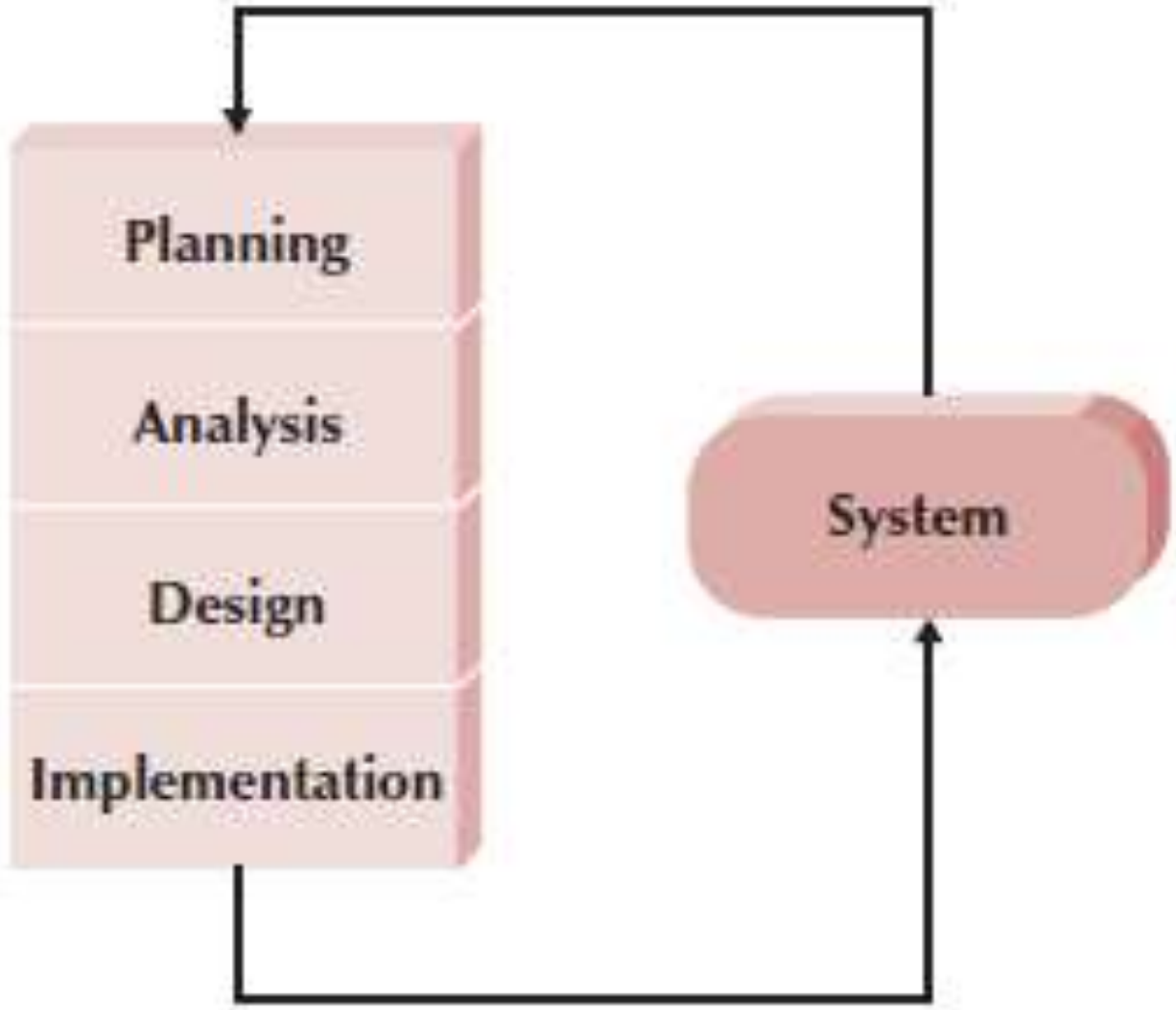
- *Individuals and interactions over processes and tools Working software over comprehensive documentation Customer collaboration over contract negotiation Responding to change over following a plan*

That is, while there is value in the items on the right, we value the items on the left more.

- Software is delivered early and continuously through the development process, satisfying the customer.
- Changing requirements are embraced regardless of when they occur in the development process.
- Working software is delivered frequently to the customer.
- Customers and developers work together to solve the business problem.
- Motivated individuals create solutions; provide them the tools and environment they need and trust them to deliver.
- Face-to-face communication within the development team is the most efficient and effective method of gathering requirements.

- The primary measure of progress is working, executing software.
- Both customers and developers should work at a pace that is sustainable. That is, the level of work could be maintained indefinitely without any worker burnout.
- Agility is heightened through attention to both technical excellence and good design.
- Simplicity, the avoidance of unnecessary work, is essential.
- Self-organizing teams develop the best architectures, requirements, and designs.
- Development teams regularly reflect on how to improve their development processes.

Typical Agile Development Methodology



How AI Itself Enables Better Methodologies

AI-Enhanced Methodology	What It Means	Example
AI-Augmented Agile	AI assists in sprint planning, story estimation, backlog prioritization	Jira with AI predicts which stories will miss the sprint
MLOps (Machine Learning Operations)	DevOps adapted for ML models – version data, models, code together	DVC (Data Version Control), MLflow
LLM-Driven Development	Use large language models to generate code, tests, documentation	GitHub Copilot, Cursor, Windsurf
Automated Methodology Selection	AI analyzes project parameters and recommends methodology	Prompt: *"Based on team size=5, unclear requirements, high risk, recommend methodology"*

Which Methodology to Use?

Usefulness for	Waterfall	Parallel	Phased	Prototyping	Throwaway Prototyping	Extreme Programming
Unclear user requirements	Poor	Poor	Good	Excellent	Excellent	Excellent
Unfamiliar technology	Poor	Poor	Good	Poor	Excellent	Poor
Complex systems	Good	Good	Good	Poor	Excellent	Poor
Reliable systems	Good	Good	Good	Poor	Excellent	Good
Short time schedule	Poor	Good	Excellent	Excellent	Good	Excellent
Schedule visibility	Poor	Poor	Excellent	Excellent	Good	Good

Key Takeaways

Learning Outcome	How AI Helped Achieve It
Compare system development methodologies	AI generated side-by-side comparisons and case study matches
Understand methodology evolution for AI projects	AI explained why traditional methodologies fail and introduced MLOps
Identify skills of a modern analyst	AI generated job descriptions and skill assessments
Articulate the AI-augmented analyst role	AI demonstrated before/after time savings and task division
Design basic AI-aware object-oriented systems	AI generated UML diagrams and explained AI-specific design patterns
Distinguish traditional OO from AI-aware OO	AI transformed traditional diagrams into AI-aware versions

The end.

Extreme programming

Perhaps the best-known and most widely used agile method.

Extreme Programming (XP) takes an ‘extreme’ approach to iterative development.

- New versions may be built several times per day;
- Increments are delivered to customers every 2 weeks;
- All tests must be run for every build and the build is only accepted if tests run successfully.

XP and Agile principles

Incremental development is supported through small, frequent system releases.

Customer involvement means full-time customer engagement with the team.

People not process through pair programming, collective ownership and a process that avoids long working hours.

Change supported through regular system releases.

Maintaining simplicity through constant refactoring of code.

Scrum

Scrum is a term that is well known to rugby fans. In rugby, a scrum is used to restart a game. In a nutshell, the creators of the Scrum method believe that no matter how much you plan, as soon as the software begins to be developed, chaos breaks out and the plans go out the window. The best you can do is to react to where the proverbial rugby ball squirts out. You then sprint with the ball until the next scrum. In the case of the Scrum methodology, a sprint lasts thirty working days. At the end of the sprint, a system is delivered to the customer.

Rolling Wave Planning

